NFT Dealers Audit Report

Version 1.0

alva-p.xyz

March 19, 2026

- Prepared by: alva-p
- Lead Auditors: Pineda Álvaro

Table of Contents

- Table of Contents
- Protocol Summary
- Disclaimer
- Risk Classification
- Audit Details
  - Scope
  - Roles
- Executive Summary
  - Issues found
- Findings
- High
- Medium
- Low
- Informational
- Gas

Protocol Summary

NFT Dealers is a NFT marketplace with pre-set supply, and resell option with `Progressive fee` 1, 3 or 5%. Collecting base price/collateral on minting. NFTs can be sold by users on any price, but the fee will grow with the resell price. Can be used for in-game events, ticketing system, no limited to any specific purpose.

Disclaimer

The alva-p team makes all effort to find as many vulnerabilities in the code in the given time period, but holds no responsibilities for the findings provided in this document. A security audit by the team is not an endorsement of the underlying business or product. The audit was time-boxed and the review of the code was solely on the security aspects of the Solidity implementation of the contracts.

Risk Classification

Table 1: Finding summary

		Impact		
		High	Medium	Low
	High	H	H/M	M
Likelihood	Medium	H/M	M	M/L
	Low	M	M/L	L

We use the CodeHawks severity matrix to determine severity. See the documentation for more details.

Audit Details

Commit Hash:

```
f34038b7b948d0902ef5ae99e9f1a4964bd3cdb5
```

Scope

```
./src/
-  MockUSDC.sol
-  NFTDealers.sol
```

Roles

There are 3 types of actors in the protocol:

Actors:

1. Owner

- deploy the smart contract and set parameters (collateral, collection name, image, symbol, etc.)
- whitelist or remove from whitelist wallets
- `reveal` the protocol

- withdraw fees

2. Whitelisted user/wallet

- mint NFT
- buy, update price, cancel listing, list NFT
- collect USDC after selling

3. Non whitelisted user/wallet

- cannot mint
- buy, update price, cancel listing, list NFT
- collect USDC after selling

Executive Summary

This report presents a security review of the `NFTDealers` smart contract. The audit was performed during the official CodeHawks review period and focused on the protocol's core flows, including minting, listing, buying, collateral handling, and fee collection.

As with any time-constrained audit, this review does not guarantee the absence of additional vulnerabilities outside those identified during the contest window. The assessment resulted in two confirmed findings: one medium-severity issue affecting the mint flow and one low-severity issue related to state inconsistency and reentrancy surface in the buy flow.

Overall, the protocol requires fixes before production use.

Issues found

Table 2: Issues found summary

Severity	Number of issues found
High	0
Medium	1
Low	1
Info	0
Total	2

Findings

Medium

[M-1] Reentrancy in mintNft() allows unexpected double execution before mint state is updated

IMPACT: LOW LIKELIHOOD: MEDIUM

Description: The `mintNft()` function performs an external call to `usdc.transferFrom(msg.sender, address(this), lockAmount)` before updating critical internal state such as `tokenIdCounter` and `collateralForMinting[tokenIdCounter]`.

```
require(usdc.transferFrom(msg.sender, address(this), lockAmount), "USDC transfer failed");

tokenIdCounter++;
collateralForMinting[tokenIdCounter] = lockAmount;
```

Impact: A malicious payment token can reenter mintNft() and trigger multiple executions before the original call updates internal accounting.

This may lead to:

- unexpected multiple mints in a single transaction
- inconsistent state transitions around `tokenIdCounter`
- unsafe reliance on external token behavior
- broader cross-function reentrancy risk if other protocol logic depends on partially updated mint state

Although the demonstrated PoC required paying the locked amount twice, the protocol still exposes a real reentrancy surface and relies on unsafe ordering of external interactions.

Proof of Concept: A malicious ERC20 was used in place of USDC. During transferFrom(), it called back into the whitelisted attacker contract, which reentered mintNft() before tokenIdCounter was incremented in the original execution.

Observed result from the test:

the initial call to `mintNft()` invoked `transferFrom()`

`transferFrom()` triggered a reentrant call to `mintNft()`

`mintNft()` executed twice in the same transaction

`totalMinted()` became 2

PoC results:

```
assertEq(nftDealers.totalMinted(), 2);
assertEq(nftDealers.collateralForMinting(1), LOCK_AMOUNT);
assertEq(nftDealers.collateralForMinting(2), LOCK_AMOUNT);
```

This confirms that mintNft() is reentrable through the external token call.

Recommended Mitigation:

1. Follow the Checks-Effects-Interactions pattern by updating internal mint state before performing external calls.
2. Also consider adding `nonReentrant` protection and using `SafeERC20`.

```
function mintNft() external payable onlyWhenRevealed onlyWhitelisted
    nonReentrant {
    if (msg.sender == address(0)) revert InvalidAddress();
    require(tokenIdCounter < MAX_SUPPLY, "Max supply reached");
    require(msg.sender != owner, "Owner can't mint NFTs");

    uint256 newTokenId = tokenIdCounter + 1;

    tokenIdCounter = newTokenId;
    collateralForMinting[newTokenId] = lockAmount;

    require(usdc.transferFrom(msg.sender, address(this), lockAmount), "
        USDC transfer failed");

    _safeMint(msg.sender, newTokenId);
}
```

[L-2] buy() exposes stale listing state during external calls, enabling reentrancy on inconsistent marketplace state

IMPACT: LOW LIKELIHOOD: MEDIUM

Description: The `buy()` function performs external interactions before fully finalizing the listing state in storage. Current flow:

```
function buy(uint256 _listingId) external payable {
    Listing memory listing = s_listings[_listingId];
    if (!listing.isActive) revert ListingNotActive(_listingId);
    require(listing.seller != msg.sender, "Seller cannot buy their own
        NFT");

    activeListingsCounter--;

```

```
    bool success = usdc.transferFrom(msg.sender, address(this), listing
        .price);
    require(success, "USDC transfer failed");
    _safeTransfer(listing.seller, msg.sender, listing.tokenId, "");

    s_listings[_listingId].isActive = false;

    emit NFT_Dealers_Sold(msg.sender, listing.price);
}
```

The issue is that `activeListingCounter` is decrementend before the external calls, while `s_listings[_listingId].isActive` is only set to false after `usdc.transferFrom(...)` and `_saferTransfer(...)` complete.

Because `_safeTransfer(...)` may invoke `onERC721Received(...)` when the buyer is a contract, external code can execute while the protocol is in an inconsistent intermediate state.

During that callback, the following condition can hold at the same time: - `totalActiveListings()` == 0 - `s_listings[_listingId].isActive` == true

This breaks the Checks-Effects-Interactions pattern and creates a real cross-function reentrancy surface where external code can observe and potentially interact with stale listing state before the purchase flow is fully finalized.

Impact: No direct theft of funds was demonstrated from this issue alone.

However, the PoC shows that `buy()` exposes an inconsistent marketplace state during external execution:

- the active listing counter is already decremented
- the corresponding listing still remains active in storage

Since no direct or indirect fund loss was proven in the demonstrated scenario, the impact is best classified as Low.

Proof of Concept:

A malicious buyer contract implementing `onERC721Received(...)` was used to observe protocol state during `_safeTransfer(...)`.

Observed execution:

1. Before calling buy(1):

- totalActiveListings() = 1
- s_listings(1).isActive = true

2. During onERC721Received(...) triggered by _safeTransfer(...):

- totalActiveListings() = 0
- s_listings(1).isActive = true

3. After buy() finishes:

- totalActiveListings() = 0
- s_listings(1).isActive = false

This proves that external code can execute while the protocol exposes stale listing state.

Illustrative test output (ty GPT for the idea of logs):

```
=== BEFORE BUY ===
totalActiveListings: 1
listing.isActive: true

=== DURING CALLBACK (observed by malicious buyer) ===
observedActiveListings: 0
observedIsActive: true

=== AFTER BUY ===
totalActiveListings: 0
listing.isActive: false
new owner: 0xF62849F9A0B5Bf2913b396098F7c7019b51A820a
```

This demonstrates that `buy()` leaves the contract in an inconsistent state during external execution and confirms the reentrancy surface reported by Slither.

Recommended Mitigation:

Apply Checks-Effects-Interactions by fully closing the listing before making external calls.

A safer structure would be:

```
function buy(uint256 _listingId) external payable nonReentrant {
    Listing memory listing = s_listings[_listingId];
    if (!listing.isActive) revert ListingNotActive(_listingId);
    require(listing.seller != msg.sender, "Seller cannot buy their own
        NFT");

    s_listings[_listingId].isActive = false;
    activeListingsCounter--;

    bool success = usdc.transferFrom(msg.sender, address(this), listing
        .price);
    require(success, "USDC transfer failed");

```

```
    _safeTransfer(listing.seller, msg.sender, listing.tokenId, "");

    emit NFT_Dealers_Sold(msg.sender, listing.price);
}
```

Additionally, using nonReentrant provides in depth against future reentrancy paths.